

(Introduction)

Dated: / /

Page No.:

Algorithm -

it is the step by step procedure that is used to solve a particular problem.

An algorithm is different from coding. it is basically a layout of a particular problem. it is not any implementation.

for e.g -

Sorting problem -

Input - A sequence of n no.s $n_1, n_2, n_3, \dots, n_n$

Output - ordering of the inputted sequence

$n_1, n_2, n_3, \dots, n_n$ such that $n_1 \leq n_2 \leq n_3 \dots \leq n_n$.

Applications of algorithm -

- (i) Websites on internet can operate only with the help of algorithms.
- (ii) The everyday electronic commerce activities are also dependent on algorithms.
- (iii) Algorithms are also used in finding shortest route in a road map.
- (iv) Algorithms are also used to implement various mathematical calculations.
- (v) Searching and sorting can be done only with the help of algorithms.

characteristics of algorithm -

(i) Input specified -

the input is the information to be changed during the calculation to create the output.

An algorithm ought to have at least 0 all around characterized inputs.

(ii) Correctness -

Every step must be logically correct.

(iii) Effectiveness -

The same algorithm can be executed on any pen and paper. It should be executable on all machines.

(iv) Finiteness -

No. of steps should be finite and result should be get in finite time.

(v) language independent -

The algorithm designed must be language-independent, i.e., it must be just plain instructions that can be implemented in any language and yet the output will be same, as expected.

→ Complexity - time and space taken by an algorithm for its completion.

Rated: / /
Page No.

Analysis of algorithms -

Analysis is the process of comparing two algorithms with respect to time, space etc.

CPU
Time

main
memory
space

Types of analysis -

- (a) Posteriori analysis
- (b) Priori analysis.

Posteriori analysis

Priori analysis

- | | |
|---|--|
| 1. it is a relative analysis. | 1. it is an absolute analysis. |
| 2. it is dependent on language of compiler and type of hardware. | 2. it is independent of language of compiler and type of hardware. |
| 3. it will give exact answer. | 3. it will give approx. answer. |
| 4. Time Complexity of an algorithm using this analysis differs from system to system. | 4. Time Complexity of an algorithm using this analysis is same for every system. |

Asymptotic notations -

Mathematical way of representing the time complexity.

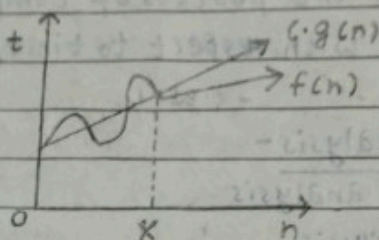
Types of asymptotic notations -

- (a) Big-oh (O)
- (b) Big-omega (Ω)
- (c) Theta (Θ)

→ little-oh - $f(n) < o(g(n))$
(0)

Date: / /
Page No.:

(a) Big-oh (O) -



→ worst case.

→ upper bound (At most time taken).

Here, $f(n) = O(g(n))$

$$f(n) \leq C \cdot g(n)$$

$$C > 0$$

$$n \geq K$$

$$K \geq 0$$

Where, $f(n)$ and $g(n)$ = functions

O = order of

C = any constant value

n = input values

K = value at any time

for e.g. - $f(n) = 2n^2 + n$

$$\text{So, } f(n) = O(g(n))$$

$$2n^2 + n \leq C \cdot n^2 \quad \text{(closest least upper bound)}$$

$$\text{So, } 2n^2 + n \leq 3n^2$$

$$n \leq n^2$$

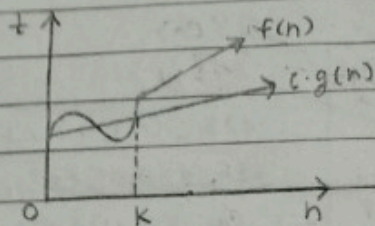
$$\boxed{n \geq 1}$$

(take it to satisfy equation)

→ little-omega - $f(n) > c \cdot g(n)$
(w)

(b)

Big-omega (Ω) -



→ Best case

→ lower bound (At least time taken)

(w) Here, $f(n) = \Omega(g(n))$

$$(w) \exists c > 0 \text{ s.t. } f(n) \geq c \cdot g(n)$$

$$\forall n \geq k$$

$$c > 0, n \geq k$$

$$k \geq 0$$

where, $f(n)$ and $g(n)$ = functions

Ω = omega of

c = any constant value

n = input values

k = value at any time.

for e.g. - $f(n) = 2n^2 + n$

so, $f(n) = \Omega(g(n))$

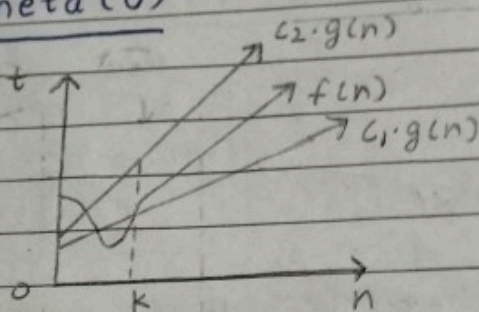
$$2n^2 + n \geq c \cdot n^2 \rightarrow \text{closest greatest lower bound}$$

$$\text{So, } 2n^2 + n \geq 2n^2$$

$$\boxed{n \geq 0}$$

(take it 2 to satisfy equation)

3.

Theta (θ) -

→ Average case

→ Exact time

$$\text{Here, } f(n) = \theta g(n) \leq c_2 g(n)$$

$$c_1 g(n) \leq f(n) \leq c_2 g(n)$$

$$c > 0$$

$$n \geq k, k \geq 0$$

where, $f(n)$ and $g(n)$ = functions θ = theta of c_1 and c_2 = any constant values n = input values k = value at any timefor e.g. - $f(n) = 2n^2 + n$

$$\text{So, } f(n) = \theta g(n)$$

$$c_1 n^2 \leq 2n^2 + n \leq c_2 n^2$$

chose acc. to equation

$$\text{So, } 2n^2 \leq 2n^2 + n \leq 3n^2$$

Comparison of various time complexities -

$$O(1) < O(\log(\log n)) < O(\log n) < O(n^{1/2}) < O(n) <$$

$$O(n \log n) < O(n^2) < O(n^3) < O(n^k) < O(2^n) <$$

$$O(n^n) < O(2^{2^n})$$

$(k \geq 4)$

$$O(1) = O(1) \rightarrow \text{fixed value}$$

In algorithms, $O(10^3) = O(10^9) = O(1) = O(1)$,
as values are constant.

O = order of (worst case)

